AI 编程三剑客：Spec-Kit、OpenSpec、Superpowers 深度对比与实战指南

本文将深入介绍 GitHub 官方的 Spec-Kit、社区热门的 OpenSpec 以及跨平台方法论工具 Superpowers 三个 AI 编程辅助工具，从安装配置到实战使用，再到三者协同的最佳实践，带你全面掌握 AI 驱动的规范化开发新范式。

前言：为什么需要这些工具？

2024-2026 年，AI 编程工具经历了爆发式增长。从最初的代码补全，到如今的 AI Agent 自主编程，开发者面临一个核心问题：如何让 AI 真正理解我们的意图，并按照预期的方式工作？

三个工具应运而生，它们从不同角度解决这个问题：

工具核心问题类比 Spec-Kit"按什么规矩干"建筑规范手册 OpenSpec"改了什么"施工变更单 Superpowers"怎么干"施工队工作手册

接下来，让我们逐一深入了解。

一、Spec-Kit：GitHub 官方的规范驱动开发框架

1.1 简介

Spec-Kit 是 GitHub 官方在 2025 年初推出的开源工具包，专为"规范驱动开发"（Spec-Driven Development）设计。它的核心理念是：先写规范，再写代码。

- GitHub 仓库：https://github.com/github/spec-kit
- Stars：69.1k ⭐
- 技术栈：Python (uv 包管理器)
- 适用 AI：Claude Code、Copilot Agent 等

1.2 核心概念

Spec-Kit 引入了分阶段的规范驱动开发流程，通过 5 个斜杠命令实现：

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                          /speckit.constitution                               │
│                       (项目宪法：全局约束、开发准则)                              │
└──────────────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                            /speckit.specify                                  │
│                        (功能规范：描述 what 和 why)                              │
└──────────────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                             /speckit.plan                                    │
│                        (技术计划：技术栈和架构选择)                              │
```

```
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                  /speckit.tasks                                                                      │
│                              （任务分解：可执行的任务清单）                                                                │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                /speckit.implement                                                                    │
│                                （执行实现：构建功能）                                                                    │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

constitution.md（宪法）：定义项目级别的治理原则
- 代码质量标准
- 测试规范
- 用户体验一致性要求
- 性能要求

spec.md（规范）：描述具体功能的需求
- 用户故事
- 功能需求
- 不涉及技术栈（关注 what 和 why）

plan.md（计划）：技术实现方案
- 技术栈选择
- 架构设计
- API 契约

tasks.md（任务）：可执行的任务清单
- 从计划中提取的具体任务
- 实现步骤

1.3 安装教程

前置条件
- Python 3.11+
- uv 包管理器
- Git
- 支持的 AI 编码助手（Claude Code、Copilot、Cursor 等）

安装步骤

```
# 1. 安装 uv（如果还没有）
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# 2. 安装 Specify CLI（持久安装，推荐）
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git

# 3. 验证安装
specify check

一次性使用（无需安装）
# 使用 uvx 直接运行
uvx --from git+https://github.com/github/spec-kit.git specify init <PROJECT_NAME>

初始化项目
# 创建新项目
specify init my-project --ai claude

# 在当前目录初始化
specify init . --ai claude
# 或使用 --here 标志
specify init --here --ai claude

# 强制初始化（跳过确认）
specify init . --force --ai claude

支持的 AI 助手:
- claude - Claude Code
- copilot - GitHub Copilot
- cursor-agent - Cursor
- gemini - Gemini CLI
- windsurf - Windsurf
- codex - Codex CLI
- opencode - opencode
- qoder - Qoder CLI
- 以及更多 20+ 工具
初始化后的目录结构:
your-project/
├── .specify/
│   ├── memory/
│   │   └── constitution.md    # 项目宪法
│   ├── scripts/               # 内置脚本
│   ├── specs/                 # 功能规范目录
│   └── templates/             # 模板文件
│       ├── plan-template.md
│       ├── spec-template.md
│       └── tasks-template.md
```

└── CLAUDE.md # AI 助手配置（根据选择的 AI 而定）

1.4 使用教程
步骤一：建立项目宪法
在 AI 助手中使用 /speckit.constitution 命令：
/speckit.constitution Create principles focused on code quality, testing standards, user experience consistency, and performance requirements

这会在 .specify/memory/constitution.md 中创建项目的治理原则。
步骤二：创建功能规范
使用 /speckit.specify 命令描述你想构建的内容（关注 what 和 why，不涉及技术栈）：
/speckit.specify Build an application that can help me organize my photos in separate photo albums. Albums are grouped by date and can be re-organized by dragging and dropping on the main page.

步骤三：创建技术计划
使用 /speckit.plan 命令提供技术栈和架构选择：
/speckit.plan The application uses Vite with vanilla HTML, CSS, and JavaScript. Images are not uploaded anywhere and metadata is stored in a local SQLite database.

步骤四：分解任务
使用 /speckit.tasks 从实现计划创建可执行的任务清单：
/speckit.tasks

步骤五：执行实现
使用 /speckit.implement 执行所有任务，按计划构建功能：
/speckit.implement

可选命令
命令描述/speckit.clarify 澄清规范中不明确的地方（推荐在 /speckit.plan 前使用）/speckit.analyze 跨工件一致性和覆盖率分析（在 /speckit.tasks 后、/speckit.implement 前使用）/speckit.checklist 生成自定义质量检查清单
示例：完整流程
假设要开发一个团队协作应用 Taskify：
1. 建立宪法
/speckit.constitution 建立代码质量、测试标准和用户体验一致性的原则

2. 定义规范
/speckit.specify Develop Taskify, a team productivity platform. It should allow users to create projects, add team members, assign tasks, comment and move tasks between boards in Kanban style.

3. 技术计划
/speckit.plan We are going to generate this using .NET Aspire,

using Postgres as the database. The frontend should use Blazor server
with drag-and-drop task boards.

4. 分解任务
/speckit.tasks

5. 执行实现
/speckit.implement

生成的文件结构：

```
.specify/
├── memory/
│   └── constitution.md
├── specs/
│   └── 001-create-taskify/
│       ├── spec.md          # 功能规范
│       ├── plan.md          # 技术计划
│       ├── tasks.md         # 任务清单
│       ├── research.md      # 技术研究
│       ├── data-model.md    # 数据模型
│       ├── quickstart.md    # 快速开始指南
│       └── contracts/
│           ├── api-spec.json    # API 契约
│           └── signalr-spec.md  # SignalR 规范
└── templates/
    ├── plan-template.md
    ├── spec-template.md
    └── tasks-template.md
```

二、OpenSpec：轻量级规范驱动开发工具

2.1 简介

OpenSpec 是由 Fission-AI 团队开发的规范驱动开发（SDD）工具，专注于灵活的、可自定义的工作流。最新版本使用 OPSX 工作流，支持 20+ AI 编码助手。

- GitHub 仓库：https://github.com/Fission-AI/OpenSpec
- Stars：23.7k ⭐
- 技术栈：TypeScript (npm)
- 适用 AI：Claude Code、Cursor、Windsurf、OpenCode、Codex、Copilot 等 20+ 工具

2.2 核心概念

OpenSpec 最新版本使用 OPSX 工作流，提供灵活的动作式工作方式，而非固定的阶段流程：

┌──┐

```
│                OPSX Workflow                  │
│            （灵活动作，迭代流动）                  │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│  /opsx:new ──────► /opsx:continue ──────► /opsx:apply ──────► /opsx:archive │
│      │            │            │            │            │
│
└──────────────────────────┴──────────────────────────┴──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘            │
│      创建工件          逐步实施             归档           │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

工作流程：
1./opsx:new - 开始新的变更（创建 proposal）
2./opsx:continue - 逐步创建工件（specs、design、tasks）
3./opsx:apply - 实施工阶段，执行任务并更新工件
4./opsx:archive - 归档完成的功能到知识库
5./opsx:explore - 探索想法，思考问题（可选）
6./opsx:ff - 快速前进，一次性创建所有规划工件
7./opsx:sync - 同步到主分支（可选）
2.3 安装教程
前置条件
- Node.js 20.19.0 或更高版本
- npm 或 pnpm（也支持 bun、yarn）
安装步骤

```
# 方式一：全局安装（推荐）
npm install -g @fission-ai/openspec@latest

# 方式二：项目级安装
npm install --save-dev @fission-ai/openspec

# 方式三：使用 npx 直接运行
npx @fission-ai/openspec init
```

初始化项目

```
# 在项目根目录运行
openspec init

# 这会创建以下结构：
# your-project/
# ├── .openspec/
# │   ├── changes/          # 活跃变更（OPSX workflow）
# │   ├── changes/archive/    # 归档的变更（知识库）
```

```
# │   ├── config.yaml        # 项目配置（可选）
# │   └── schemas/          # 自定义工作流模式（可选）
# └── .claude/skills/openspec-*  # 自动生成的技能
```

提示：初始化时会提示创建 openspec/config.yaml 项目配置文件，这是可选但推荐的。

2.4 使用教程

步骤一：创建配置文件（可选）

```
# openspec/config.yaml
schema: spec-driven

context: |
  Tech stack: TypeScript, React, Node.js
  API conventions: RESTful, JSON responses
  Testing: Vitest for unit tests, Playwright for e2e
  Style: ESLint with Prettier, strict TypeScript

rules:
  proposal:
    - Include rollback plan
    - Identify affected teams
  specs:
    - Use Given/When/Then format for scenarios
  design:
    - Include sequence diagrams for complex flows
```

配置说明：

- schema：默认工作流模式（当前为 spec-driven）
- context：项目上下文，会注入到所有工件
- rules：每个工件的具体规则

步骤二：创建新变更

```
# 开始新的变更
/opsx:new Add user profile page
```

AI 会询问：

1.你想构建什么？

2.使用哪个工作流模式？

生成的工件结构：

```
openspec/changes/add-user-profile-page/
├── proposal.md        # 变更提案（为什么、范围、方法）
├── specs/             # 功能规范
│   └── spec.md
├── design.md          # 技术设计
└── tasks.md           # 实施任务清单
```

步骤三：逐步创建工件
继续创建下一个工件（基于依赖关系）
/opsx:continue

每次调用会：
1.检查哪些工件已准备好
2.创建一个工件
3.显示解锁的下一个工件
步骤四：快速前进
一次性创建所有规划工件
/opsx:ff add-user-profile-page

使用场景：当你已经清楚要构建什么，想要快速启动时。
步骤五：实施
执行任务，并更新工件
/opsx:apply

AI 会：
1.遍历 tasks.md 中的任务
2.逐一实现
3.实时更新任务状态
4.如有问题，更新 specs/ 或 design/
步骤六：归档
完成后归档
/opsx:archive add-user-profile-page

将变更移动到知识库：
openspec/changes/archive/2025-02-12-add-user-profile-page/

步骤七：探索想法
不确定要构建什么时，先探索
/opsx:explore

这是一个思考伙伴，帮助澄清想法、比较选项、明确需求。
查看状态
查看当前变更状态
openspec status --change add-user-profile-page --json

返回：
{
  "artifacts": [
    {"id": "proposal", "status": "done"},
    {"id": "specs", "status": "ready"},
    {"id": "design", "status": "ready"},

```
    {"id": "tasks", "status": "in_progress"}
  ]
}
```

三、Superpowers：Claude Code 的"施工队工作手册"

3.1 简介

Superpowers 是由 Jesse Vincent（obra）开发的 AI 编程方法论工具包，专注于执行方法论。它不是文档管理工具，而是一套让 AI 更高效、更可靠地执行编码任务的最佳实践集合，通过"技能"（Skills）系统引导 AI 像高级工程师一样工作。

- GitHub 仓库：https://github.com/obra/superpowers
- Stars：50k ⭐
- 技术栈：Markdown + JavaScript Plugin
- 适用 AI：Claude Code、OpenCode、Codex

3.2 核心概念

Superpowers 的核心是 **"让 AI 像高级工程师一样工作"**：

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              Superpowers 方法论                              │
├──────────────────────────────────────────────────────────────────────────────┤
│  🧪 TDD-First        │ 强制 AI 先写测试，再写实现                            │
├──────────────────────────────────────────────────────────────────────────────┤
│  🤖 Sub-Agents       │ 拆分复杂任务给专门的子代理                            │
├──────────────────────────────────────────────────────────────────────────────┤
│  📝 Code Review      │ 实现后自动触发代码审查                                │
├──────────────────────────────────────────────────────────────────────────────┤
│  🔍 Exploration      │ 实现前先充分探索代码库                                │
├──────────────────────────────────────────────────────────────────────────────┤
│  ✅ Verification     │ 每步都要验证，不盲目前进                              │
└──────────────────────────────────────────────────────────────────────────────┘
```

3.3 安装教程

前置条件

- Bash 或 Zsh shell (macOS/Linux)
- PowerShell 或 Command Prompt (Windows)

Claude Code 用户（推荐方式）

Claude Code 有插件市场，安装最简单：

```
# 在 Claude Code 中执行
```

```
/plugin marketplace add obra/superpowers-marketplace
/plugin install superpowers@superpowers-marketplace

# 验证安装
/help
```

看到 brainstorm、write-plan、execute-plan 这 3 个命令就说明安装成功。

OpenCode 用户

```
# 1. 克隆 Superpowers 仓库
git clone https://github.com/obra/superpowers.git ~/.config/opencode/superpowers

# 2. 创建目录
mkdir -p ~/.config/opencode/plugins ~/.config/opencode/skills

# 3. 创建符号链接（插件）
ln -s ~/.config/opencode/superpowers/.opencode/plugins/superpowers.js ~/.config/opencode/plugins/superpowers.js

# 4. 创建符号链接（技能）
ln -s ~/.config/opencode/superpowers/skills ~/.config/opencode/skills/superpowers

# 5. 重启 OpenCode
```

Windows 用户（PowerShell）

```
# 1. 克隆仓库
git clone https://github.com/obra/superpowers.git "$env:USERPROFILE\.config\opencode\superpowers"

# 2. 创建目录
New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\.config\opencode\plugins"
New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\.config\opencode\skills"

# 3. 创建符号链接（插件，需要开发者模式或管理员权限）
New-Item -ItemType SymbolicLink -Path "$env:USERPROFILE\.config\opencode\plugins\superpowers.js" -Target "$env:USERPROFILE\.config\opencode\superpowers\.opencode\plugins\superpowers.js"

# 4. 创建符号链接（技能，无需特殊权限）
New-Item -ItemType Junction -Path "$env:USERPROFILE\.config\opencode\skills\superpowers" -Target "$env:USERPROFILE\.config\opencode\superpowers\skills"
```

Git Bash 用户
Git Bash 的 ln 命令会复制文件，需使用 cmd //c
mkdir -p ~/.config/opencode/plugins ~/.config/opencode/skills

cmd //c "mklink \"$(cygpath -w ~/.config/opencode/plugins/superpowers.js)\" \"$(cygpath -w ~/.config/opencode/superpowers/.opencode/plugins/superpowers.js)\""

cmd //c "mklink /J \"$(cygpath -w ~/.config/opencode/skills/superpowers)\" \"$(cygpath -w ~/.config/opencode/superpowers/skills)\""

更新 Superpowers
OpenCode 用户
cd ~/.config/opencode/superpowers
git pull

Claude Code 用户（如果有安装）
cd ~/.claude/superpowers
git pull

卸载
OpenCode 用户
rm ~/.config/opencode/plugins/superpowers.js
rm -rf ~/.config/opencode/skills/superpowers

可选：删除源码
rm -rf ~/.config/opencode/superpowers

看到 brainstorm、write-plan、execute-plan 这 3 个命令就说明安装成功。
OpenCode 用户
OpenCode 需要手动配置：
1. 克隆 Superpowers 仓库
git clone https://github.com/obra/superpowers.git ~/.config/opencode/superpowers

2. 创建目录
mkdir -p ~/.config/opencode/plugins ~/.config/opencode/skills

3. 创建符号链接（插件）
ln -s ~/.config/opencode/superpowers/.opencode/plugins/superpowers.js ~/.config/opencode/plugins/superpowers.js

4. 创建符号链接（技能）
ln -s ~/.config/opencode/superpowers/skills ~/.config/opencode/skills/superpowers

5. 重启 OpenCode

6. 验证安装
在 OpenCode 中问："Do you have superpowers?"

Windows 用户（PowerShell）：
1. 克隆仓库
git clone https://github.com/obra/superpowers.git "$env:USERPROFILE\.config\opencode\superpowers"

2. 创建目录
New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\.config\opencode\plugins"
New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\.config\opencode\skills"

3. 创建插件符号链接（需要开发者模式或管理员权限）
New-Item -ItemType SymbolicLink -Path "$env:USERPROFILE\.config\opencode\plugins\superpowers.js" -Target "$env:USERPROFILE\.config\opencode\superpowers\.opencode\plugins\superpowers.js"

4. 创建技能目录链接（无需特殊权限）
New-Item -ItemType Junction -Path "$env:USERPROFILE\.config\opencode\skills\superpowers" -Target "$env:USERPROFILE\.config\opencode\superpowers\skills"

5. 重启 OpenCode

Codex 用户
1. 克隆仓库
git clone https://github.com/obra/superpowers.git ~/.codex/superpowers

2. 创建符号链接
mkdir -p ~/.agents/skills
ln -s ~/.codex/superpowers/skills ~/.agents/skills/superpowers

3. 重启 Codex

Windows 用户（PowerShell）：
1. 克隆仓库
git clone https://github.com/obra/superpowers.git "$env:USERPROFILE\.codex\superpowers"

2. 创建符号链接
New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\.agents\skills"
New-Item -ItemType Junction -Path "$env:USERPROFILE\.agents\skills\superpowers" -Target "$env:USERPROFILE\.codex\superpowers\skills"

3. 重启 Codex

更新 Superpowers
OpenCode 用户
cd ~/.config/opencode/superpowers && git pull

Codex 用户
cd ~/.codex/superpowers && git pull

卸载
OpenCode 用户
rm ~/.config/opencode/plugins/superpowers.js
rm -rf ~/.config/opencode/skills/superpowers
可选：删除源码
rm -rf ~/.config/opencode/superpowers

Codex 用户
rm ~/.agents/skills/superpowers
可选：删除源码
rm -rf ~/.codex/superpowers

3.4 使用教程
Superpowers 工作原理
Superpowers 提供两种工作方式：
1.Commands（快捷命令）：3 个可用命令
- brainstorm - 头脑风暴
- write-plan - 编写计划
- execute-plan - 执行计划
2.Skills（技能系统）：AI 根据上下文自动加载合适的技能，无需记忆命令名称
推荐使用：自然语言描述需求，让 Superpowers 自动选择最合适的方式。
核心技能列表
技能名称用途 brainstorming 在任何创造性工作前先头脑风暴，理解需求后再动手 subagent-driven-development 子代理驱动开发：为每个任务派发独立子代理 + 两阶段审查 executing-plans 执行已制定的实施计划 finishing-a-development-branch 完成开发分支：合并、创建 PR 或保留 requesting-code-review 请求代码审查 receiving-code-review 接收并处理代码审查反馈 systematic-debugging 系统化调试：解决 bug 时使用 test-driven-developmentTDD 工作流：测试驱动开发 using-git-worktrees 使用 Git Worktree 创建隔离的工作空间 verification-before-completion 完成前验证：每步都要验

证 writing-plans 编写实施计划 writing-skills 编写自定义技能
技能触发机制

```
┌──────────────
```

- 每节确认是否正确
- 不对则回溯修改
- Step 5: 生成设计文档
- 保存到 docs/plans/YYYY-MM-DD-coupon-design.md
- 提交到 git
2. 子代理驱动开发
当有明确的实现计划后：
"帮我实施优惠券功能，按照上面的设计文档"

Superpowers 的 subagent-driven-development 技能会启动：
- 提取所有任务到 TodoWrite
- 为每个独立任务派发子代理
🤖 Sub-Agent 1: 实现优惠券模型
  - 编写测试（TDD）
  - 实现代码
  - 提交并自审查

🤖 Sub-Agent 2: 实现认证服务
  - 编写测试（TDD）
  - 实现代码
  - 提交并自审查

🤖 Sub-Agent 3: 实现 API 端点
  - 编写测试（TDD）
  - 实现代码
  - 提交并自审查

🤖 Spec Reviewer: 验证是否符合规范
  - 检查代码是否匹配设计文档

🤖 Code Quality Reviewer: 代码质量审查
  - 检查代码质量、安全性、性能

📋 标记任务完成

3. 请求代码审查
"请帮我审查一下这段代码"

Superpowers 的 requesting-code-review 技能会：
- 分析代码变更
- 从多个角度审查：
- 代码质量
- 安全性
- 性能

- 测试覆盖率
- 文档完整性
- 提供具体建议
- 使用友好的、建设性的语气
4. 系统化调试
"用户登录时出现 500 错误"

Superpowers 的 systematic-debugging 技能会：
- 复现问题
- 分析相关代码
- 定位根本原因
- 提出修复方案
- 验证修复
示例 2：子代理驱动开发
当有明确的实现计划后：
说出：
"帮我实施优惠券功能，按照上面的设计文档"

Superpowers 的 subagent-driven-development 技能会启动：
#
📋 Step 1: 读取计划，提取任务
- 读取设计文档
- 提取所有任务
- 创建 TodoWrite 任务列表
#
🤖 Step 2: 对每个任务执行以下循环：
#
a) 派发实现者子代理
"实现优惠券模型"
子代理独立工作：实现 + 测试 + 提交 + 自审查
#
b) 派发规范审查子代理
"检查代码是否符合设计文档"
如果不符合 -> 返回 a) 修复
#
c) 派发代码质量审查子代理
"检查代码质量、安全性、性能"
如果有问题 -> 返回 a) 修复
#
d) 标记任务完成
#
🔍 Step 3: 重复直到所有任务完成
#
✅ Step 4: 最终整体审查

派发最终代码审查子代理
#
🌳 Step 5: 完成分支
使用 finishing-a-development-branch 技能

示例 3：请求代码审查
在提交代码前：
"请帮我审查一下这段代码"

Superpowers 的 requesting-code-review 技能会：
#
1. 分析代码变更
2. 从多个角度审查：
- 代码质量
- 安全性
- 性能
- 测试覆盖率
- 文档完整性
3. 提供具体建议
4. 使用友好的、建设性的语气

示例 4：使用 Git Worktree 隔离开发
在开始新功能开发前：
"我要开发优惠券功能，帮我创建一个隔离的工作空间"

Superpowers 的 using-git-worktrees 技能会：
#
1. 检查当前 git 状态
2. 创建新分支：feature/coupon-system
3. 创建 Git Worktree: .worktrees/feature-coupon-system/
4. 更新项目配置（如果需要）
5. 切换到新工作空间
#
这样可以：
- 保持主分支干净
- 多个功能并行开发互不干扰
- 快速切换上下文

技能触发机制
Superpowers 的技能会根据上下文自动触发：

```
┌──────────────
```

```
──────────────────────────────────────┘
                  │
                  ▼
          ┌──────────────────────────────────────┐
          │  分析请求类型          │
          └──────────────────────────────────────┘
                  │
      ┌───────────────────────────┼───────────────────────────┐
      ▼           ▼           ▼
   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
   │  新想法    │   │有计划    │   │要审查    │
   │头脑风暴   │   │实施任务  │   │代码质量  │
   └──────────────────┘   └──────────────────┘   └──────────────────┘
      │           │           │
      ▼           ▼           ▼
  brainstorming  subagent-driven  requesting
              development     code-review
```

你不需要记住所有技能名称，只需说出你的需求，Superpowers 会自动选择合适的技能。

3.5 自定义技能

你可以创建自己的技能来扩展 Superpowers 的能力。

项目级技能

在项目的 .opencode/skills/ 目录下创建自定义技能：

```
# 创建技能目录
mkdir -p .opencode/skills/my-custom-skill

# 创建技能文件
cat > .opencode/skills/my-custom-skill/SKILL.md << 'EOF'
---
name: my-custom-skill
description: "Use when [condition] - [what it does]"
---

# My Custom Skill

## When to Use

Use this skill when...

## The Process

1. First step...
2. Second step...
3. Third step...
```

```
## Key Principles

- Principle 1
- Principle 2
EOF
```

个人级技能（OpenCode）
在 ~/.config/opencode/skills/ 目录下创建：

```
mkdir -p ~/.config/opencode/skills/my-personal-skill

# 创建技能文件
cat > ~/.config/opencode/skills/my-personal-skill/SKILL.md << 'EOF'
---
name: my-personal-skill
description: "My personal coding preferences"
---

# My Personal Coding Preferences

## Code Style
- Use functional components only (no class components)
- Prefer `const` over `let`
- Use named exports, not default exports

## Testing
- Minimum 80% coverage for new code
- Use React Testing Library, not Enzyme
- Mock external APIs in tests

## Git
- Use conventional commits
- Squash before merge
- No force push to main
EOF
```

技能优先级
Superpowers 会按以下优先级加载技能：
1.项目级技能（.opencode/skills/）- 最高优先级
2.个人级技能（~/.config/opencode/skills/）- 中优先级
3.Superpowers 技能（~/.config/opencode/skills/superpowers/）- 默认技能
这意味着你可以：
- 在项目中覆盖默认行为
- 为不同项目定制不同的规则

- 保持个人编码偏好的一致性
实战示例：创建团队技能
假设你的团队有特定的数据库规范：
---
name: team-database-rules
description: "Must use for all database operations"

Team Database Rules

When to Use

Use this skill when:
- Creating new database models
- Writing SQL queries
- Designing database schemas
- Writing migrations

The Process

1. Schema Design
- Always use snake_case for column names
- Include `created_at` and `updated_at` timestamps
- Add appropriate indexes for foreign keys

2. Query Writing
- Use parameterized queries only
- Never concatenate strings for SQL
- Add `EXPLAIN` analysis for complex queries

3. Migrations
- Always write rollback migration
- Test migration on staging first
- Back up production schema before deploying

Key Principles

- Performance first: optimize for query speed
- Safety second: prevent SQL injection
- Maintainability third: clear naming and documentation

将此技能放在 .opencode/skills/team-database-rules/SKILL.md，整个团队都会自动遵守这些规则。

四、三者对比分析

4.1 核心对比表

⚠ 重要提示：Spec-Kit 和 OpenSpec 都是"规范驱动开发"（SDD）工具，解决的是同一个问题——防止 AI "vibe coding" 导致的实现漂移。两者是竞争关系，应该二选一，而不是同时使用。Superpowers 则是执行方法论工具，与两者互补。

维度Spec-KitOpenSpecSuperpowers维护方GitHub 官方Fission-Alobra（社区)Stars69.1k23.7k50k技术栈Python (uv)TypeScript (npm)Markdown + JS Plugin工具类型🔵 规范管理（SDD）🔵 规范管理（SDD）🟢 执行方法论核心理念分阶段规范驱动统一真相源 + 增量变更TDD + 代码审查规范结构分散式（每功能独立文件）统一式（单一文档）无规范管理迭代速度较慢（严格阶段流程）较快（轻量级循环）N/A适用项目新项目 / 复杂系统存量项目 / 快速迭代所有项目与其他工具关系⚔ 与 OpenSpec 竞争⚔ 与 Spec-Kit 竞争✅ 与两者互补

4.2 工作流对比

⚠ 重要提示：Spec-Kit 和 OpenSpec 的工作流功能高度重叠，都是从"意图捕获"到"任务分解"再到"实现"的完整链路。选择其一即可，不建议同时使用。

```
┌──────────────
```

```
│              ┌──────────────────────┐              ┌──────────────────────┐
 ┌──────────────────────┐                │
│   │ /opsx:new   │ -> │ /opsx:      │ -> │  /opsx:apply │                  │
│   │             │    │  continue   │    │              │                  │
│              └──────────────────────┘              └──────────────────────┘
 └──────────────────────┘                │
│      │            │            │                      │
│      ▼            ▼            ▼                      │
│   创建提案       迭代细化       应用到代码                    │
│      │                       │                       │
│
└──────────────────────────────────────────────────────────────┼──────────────
──────> /opsx:archive (归档)  │
│                                                      │
│  📁 产出物：spec.md (统一规范文件) + archived/ (历史归档)            │
│  🎯 特点：可随时跳转，灵活迭代，单一真相源                          │
│                                                      │
└──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────────┐
│                 执行方法论工具（与上述任一搭配）                    │
├──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────────┤
│                                                      │
│  Superpowers 工作流（TDD 螺旋、AI 执行最佳实践）:                    │
│                                                      │
│          ┌──────────────────────────────────────────────────────────────┐
                │
│          │                      │                  │
│          ▼                      │                  │
│     ┌──────────────────┐    ┌──────────────────┐    ┌──────────────────┐
          │
│     │   探索    │ -> │写测试    │ -> │   实现    │                  │
│     └──────────────────┘    └──────────────────┘    └──────────────────┘
          │
│                 │          │                    │
│                 ▼          │                    │
│              ┌──────────────────┐        │                    │
│              │运行测试  │ ────────────────┘                    │
│              └──────────────────┘   失败则迭代                    │
│                 │                              │
│                 ▼ 通过                          │
```

```
│                ┌──────────────┐                          │
│                │代码审查  │                          │
│                └──────────────┘                          │
│                                                          │
│  🎯 特点：与 SDD 工具正交，专注"怎么高质量实现"，而非"实现什么"        │
│                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

关键区别总结：
维度 Spec-KitOpenSpec 规范结构分散式（每功能独立目录）统一式（单一 spec.md）阶段控制严格顺序（必须逐阶段推进）灵活跳转（可随时切换）适合场景 Greenfield（从零开始）Brownfield（存量改造）迭代速度较慢（完整流程保障质量）较快（轻量级快速循环）团队规模大团队（需要流程规范）小团队/个人（灵活优先）
4.3 适用场景对比（取舍指南）

💡 核心原则：Spec-Kit 和 OpenSpec 二选一，再搭配 Superpowers 作为执行方法论。
选择 Spec-Kit 的场景
场景为什么选 Spec-Kit🏗️ 新项目从零开始分阶段流程确保基础设计不被跳过🏦 金融/医疗/合规项目严格的阶段文档满足审计要求👥 大型团队协作阶段门控防止不同成员各自为战📋 复杂系统架构 constitution.md 保证全局一致性🎯 需要完整设计文档自动生成 specify/plan/tasks 文档链
选择 OpenSpec 的场景
场景为什么选 OpenSpec🔧 存量项目改造无需从头建立完整规范体系🚀 初创公司快速迭代轻量级循环不拖慢开发节奏🧪 原型/MVP 开发快速验证想法，规范可后补👤 个人/小团队项目单一 spec.md 便于维护🔄 频繁需求变更灵活跳转适应变化
Superpowers：无论选哪个都要用
组合方案效果 Spec-Kit + Superpowers 严格规范 + TDD 执行 = 企业级质量保障 OpenSpec + Superpowers 灵活规范 + TDD 执行 = 敏捷高质量交付仅 Superpowers 无规范管理，但 AI 执行质量有保障（适合简单任务）
❌ 不推荐的组合
组合为什么不推荐 Spec-Kit + OpenSpec 功能重叠，两套规范系统会造成混乱仅 Spec-Kit 或仅 OpenSpec 缺少执行方法论，AI 可能"乱写代码"三者都不用"Vibe Coding" 模式，小项目可行，复杂项目必翻车

五、协同方案：两种最佳实践路径

⚠ 重要：由于 Spec-Kit 和 OpenSpec 功能重叠，本章提供两个独立方案，根据项目情况选择其一。
5.1 协同架构（二选一）

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                                          │
│    方案 A：Spec-Kit + Superpowers（推荐：新项目、复杂系统、大团队）        │
```

```
│                                                │
│
┌──────────────
```

```
┌──────────────
```

└──┘ │
│ │
└──┘

方案对比：
维度方案 A (Spec-Kit + Superpowers)方案 B (OpenSpec + Superpowers)启动成本较高（需建立 constitution）较低（直接 /opsx:new）规范严格度高（阶段门控）中（灵活循环）迭代速度较慢（完整流程）较快（轻量级）文档产出丰富（多层文档）精简（单一 spec.md）适合团队大团队、跨职能协作小团队、个人开发适合项目新项目、复杂系统存量项目、快速验证
5.2 实战工作流
假设我们要为一个电商项目添加"优惠券系统"。下面分别演示两种方案的完整工作流。

🅐 方案 A：Spec-Kit + Superpowers（新项目/复杂系统）
适用场景：从零开始的电商项目，需要完整设计文档和严格流程控制。
Step 1：建立项目宪法
```
# 创建项目治理原则（只需执行一次，全项目共享）
/specify:constitution

# AI 会引导你定义：
# - 代码质量标准
# - 测试覆盖要求
# - 安全约束
# - 性能要求
```

生成的 .specify/memory/constitution.md：
```
## Project Constitution

### Code Quality
- TypeScript strict mode required
- No `any` types allowed
- 80% test coverage minimum

### Security
- All user inputs must be validated
- Coupon codes case-insensitive, prevent timing attacks
- Rate limiting on validation endpoints

### Performance
- Coupon validation < 50ms
- Use Redis caching for hot paths
```

Step 2：定义功能规范
定义"做什么"和"为什么"
/specify:specify Implement a coupon system with creation, validation, and application for the e-commerce platform.

生成的 .specify/coupon-system/specify.md：
Coupon System Specification

Goal
Enable promotional discounts through a flexible coupon system.

Requirements
- Create, read, update, delete coupons
- Validate coupon applicability
- Apply discount to orders
- Track usage statistics

Constraints
- Must comply with constitution.md security requirements
- Coupons cannot be combined unless explicitly allowed

Step 3：制定技术方案
定义"怎么做"
/specify:plan Use PostgreSQL for storage, Redis for caching. REST API with TypeScript/Express.

生成的 .specify/coupon-system/plan.md：
Technical Plan

Architecture
- CouponService: Core business logic
- CouponRepository: PostgreSQL persistence
- CouponCache: Redis caching layer
- CouponController: REST API endpoints

Data Model
- coupons table: id, code, discount_type, value, valid_from, valid_until
- coupon_usages table: id, coupon_id, order_id, used_at

Step 4：分解任务
生成可执行任务清单
/specify:tasks

生成的 .specify/coupon-system/tasks.md：

Implementation Tasks

- [ ] Create Coupon entity and migration
- [ ] Implement CouponRepository
- [ ] Implement CouponCache with Redis
- [ ] Implement CouponService with validation logic
- [ ] Create REST API endpoints
- [ ] Add integration tests
- [ ] Add E2E tests

Step 5：执行实现（+ Superpowers TDD）
开始实现（自动触发 Superpowers TDD 循环）
/specify:implement

Superpowers 自动介入：
1.探索阶段（brainstorming）
- 确认技术方案细节
- 识别潜在风险
2.TDD 循环（每个任务）// 先写测试 describe('CouponService', () => { it('should validate active coupon', async () => { const coupon = await createTestCoupon({ code: 'TEST20' }); const result = await couponService.validate('TEST20'); expect(result.valid).toBe(true); }); it('should reject expired coupon', async () => { const result = await couponService.validate('EXPIRED'); expect(result.valid).toBe(false); expect(result.reason).toBe('COUPON_EXPIRED'); });});// 再写实现// 运行测试验证
3.代码审查（code-review）
- 检查是否符合 constitution.md 约束
- 验证测试覆盖率

🅱 方案 B：OpenSpec + Superpowers（存量项目/快速迭代）
适用场景：已有电商项目，需要快速添加优惠券功能，不需要完整设计文档。
Step 1：创建规范提案
快速创建功能提案
/opsx:new Add coupon system with percentage and fixed discounts

生成的 spec.md：
Coupon System Proposal

Summary
Add promotional coupon functionality to existing e-commerce platform.

Scope
- Coupon CRUD operations
- Validation logic

- Order integration

Out of Scope
- Complex stacking rules (future iteration)
- A/B testing integration

Step 2：迭代细化
根据反馈迭代规范
/opsx:continue Add Redis caching requirement and rate limiting

更新后的 spec.md：
Coupon System Proposal

Summary
Add promotional coupon functionality to existing e-commerce platform.

Technical Decisions
- Use existing PostgreSQL for storage
- Add Redis caching for validation performance
- Rate limit: 10 validations/minute per user

Implementation Notes
- Reuse existing validation middleware
- Follow current API conventions (REST, JSON responses)

Step 3：应用到代码（+ Superpowers TDD）
开始实现
/opsx:apply

Superpowers 自动介入（与方案 A 相同的 TDD 循环）：
1.探索阶段 → 确认现有代码结构
2.TDD 循环 → 先写测试，再实现
3.代码审查 → 确保符合现有代码风格
Step 4：归档
功能完成后归档
/opsx:archive

spec.md 移动到 archived/coupon-system-2024-01.md。

两种方案对比总结
步骤方案 A (Spec-Kit)方案 B (OpenSpec)建立全局规范✅ constitution❌ 跳过功能定义 specify (详细)new (简洁)技术方案 plan (独立文档)continue (迭代到同一文件)任务分解 tasks（自动生成)apply 时自动分解执行实现 implement + Superpowersapply + Superpowers 归档文档保留在 .specify/archive 到历史目录总耗时较长（完整流程）较短

（快速循环）文档质量高（多层结构化文档）中（单一迭代文档）
5.3 配置集成
根据你选择的方案，配置相应的 Claude Rules 文件：
方案 A 配置：Spec-Kit + Superpowers

```
<!-- ~/.claude/rules/spec-kit-integration.md -->

# Spec-Kit + Superpowers Integration

## Before Any Implementation Work

1. ALWAYS check `.specify/memory/constitution.md` for global constraints
2. Read the current feature's spec files in `.specify/<feature>/`
3. Verify all tasks in `tasks.md` before starting

## During Implementation

1. Follow Superpowers TDD workflow for each task:
   - brainstorm → write-tests → implement → run-tests → code-review
2. Ensure all CONSTITUTION constraints are met
3. Update implementation notes in relevant spec files

## After Implementation

1. Run Superpowers code review
2. Verify against spec acceptance criteria
3. Mark completed tasks in `.specify/<feature>/tasks.md`

## ⚠ Do NOT

- Skip the specification phase
- Implement without checking constitution.md
- Merge code that violates defined constraints
```

方案 B 配置：OpenSpec + Superpowers

```
<!-- ~/.claude/rules/openspec-integration.md -->

# OpenSpec + Superpowers Integration

## Before Any Implementation Work

1. Check if active `spec.md` exists
2. If not, create one with `/opsx:new`
3. Review `archived/` for similar past implementations
```

During Implementation

1. Follow Superpowers TDD workflow:
   - brainstorm → write-tests → implement → run-tests → code-review
2. Update `spec.md` with implementation decisions using `/opsx:continue`
3. Keep spec in sync with actual implementation

After Implementation

1. Run Superpowers code review
2. Archive completed spec with `/opsx:archive`
3. Document learnings for future reference

⚠ Do NOT

- Implement without a spec.md
- Let spec.md drift from actual implementation
- Skip the archive step after completion

Superpowers 通用配置（两种方案都需要）
<!-- ~/.claude/rules/superpowers-config.md -->

Superpowers TDD Configuration

Mandatory Workflow

For EVERY implementation task:

1. **Exploration First**
   - Run brainstorming skill before coding
   - Understand existing codebase patterns
   - Identify potential conflicts

2. **TDD Cycle**
   - Write failing test FIRST
   - Implement minimum code to pass
   - Refactor if needed
   - Target 80%+ coverage

3. **Code Review**
   - Run code-review skill after implementation
   - Address all CRITICAL and HIGH issues
   - Document any accepted MEDIUM issues

Quality Gates

- [ ] All tests passing
- [ ] No TypeScript errors
- [ ] Code review completed
- [ ] Spec/spec.md updated

六、总结与建议

6.1 工具选择指南（决策树）

```
┌──────────────
```

```
└── ❌ 不推荐的选择
    ├── Spec-Kit + OpenSpec（功能重叠，两套规范会混乱）
    ├── 仅 Spec-Kit/OpenSpec（缺少执行方法论）
    └── 三者都不用（"Vibe Coding"，复杂项目必翻车）
```

6.2 核心洞察：理解工具定位

```
┌──────────────
```

6.3 学习路径建议

阶段 1：立即提升（Day 1）

```
──────────────────────────────────────
└── 安装 Superpowers
```

- 零配置，5 分钟上手
- 立即体验 TDD 和代码审查
- 即使不用规范工具也能提升 AI 输出质量

阶段 2：规范入门（Week 1）

```
──────────────────────────────────────
└── 选择并学习一个 SDD 工具
    ├── 新项目/严格要求 → Spec-Kit
    └── 存量项目/灵活迭代 → OpenSpec
```

- 在一个真实项目中完整走一遍流程
- 理解"规范先行"的价值

阶段 3：形成习惯（Month 1）

```
──────────────────────────────────────
└── 建立个人/团队工作流
```

- 配置 Claude Rules 自动化流程
- 积累项目特定的 constitution/spec 模板
- 从被动使用到主动依赖

6.4 常见误区

误区正确认识"三个工具一起用效果最好"❌ Spec-Kit 和 OpenSpec 功能重叠，同时用会造成混乱"规范工具会拖慢开发速度"✅ 前期投入换来后期少返工，复杂项目 ROI 极高"简单项目不需要规范"⚠️ 取决于"简单"的定义，Superpowers 单独用也行"AI 够聪明不需要约束"❌ AI 会"Vibe Coding"，规范是防止漂移的护栏

6.5 未来展望

AI 编程工具正在从"代码补全"向"自主工程"演进。

当前阶段：我们需要 Spec-Kit/OpenSpec 这样的规范工具来"约束" AI，防止它凭感觉乱写代码。Superpowers 这样的方法论工具确保 AI 按照工程最佳实践执行。

未来趋势：

- 规范工具会趋同：Spec-Kit 和 OpenSpec 解决同一问题，未来可能合并或出现更优方案
- 执行方法论会内化：TDD、代码审查等实践可能被 AI 原生支持
- 人机协作会深化：从"人写规范，AI 执行"到"人审批，AI 全程"

现在开始的意义：

- 培养"规范驱动"的思维习惯
- 积累与 AI 高效协作的经验
- 为更自主的 AI 工程时代做准备

💡 核心理念：让 AI 成为可靠的工程伙伴，而非需要时刻看管的实习生。

参考资源

官方资源

- Spec-Kit GitHub
- OpenSpec GitHub
- Superpowers GitHub
- GitHub Blog: Spec-Driven Development

深度对比分析

- Spec-Kit vs OpenSpec 深度对比 - 🌟 推荐阅读，本文核心观点来源
- OpenSpec vs Spec Kit: Hashrocket 对比

入门教程

- Superpowers 入门指南
- OpenSpec OPSX 工作流文档
- OpenSpec CLI 文档
- OpenSpec Supported Tools 文档
- Superpowers OpenCode 安装文档

如果这篇文章对你有帮助，欢迎点赞、收藏、关注！有问题欢迎在评论区讨论 🚀